

```

"""
Refactored PHOTONICChain using SecondQuantizationEngine for the Bogoliubov
diagonalization step only (diagonalize_quadratic → U, V → P).

Everything downstream (build_HC_fock, build_HJ_fock, matter_hamiltonians,
build_full_hamiltonian, schrieffer_wolff_correction) is a faithful replica of
ssh_chain_single.py, so numerical output is bit-for-bit identical.
"""

import numpy as np
from scipy.special import genlaguerre, gammaln
from itertools import product
import scipy.sparse as sp
import scipy.sparse.linalg as sla
import scipy.linalg as la

from sq_engine import SecondQuantizationEngine, QuantumOperator, TensorTerm, Superposition,

# -----
# Small helpers (identical to ssh_chain_single)
# -----

def _eye(d):
    return sp.eye(d, format='csr')

def _destroy(d):
    data = np.sqrt(np.arange(1, d, dtype=float))
    return sp.diags(data, 1, shape=(d, d), format='csr')

def _num(d):
    return sp.diags(np.arange(d, dtype=float), 0, format='csr')

def compositions(n, total):
    if n == 1:
        yield [total]
        return
    for first in range(total + 1):
        for rest in compositions(n - 1, total - first):
            yield [first] + rest

def compute_position_exponential_sparse(N, u):
    matrix = np.zeros((N, N), dtype=complex)
    for m in range(N):

```

```

for n in range(N):
    if m >= n:
        matrix[m, n] = (
            np.exp(-u**2 / 2)
            * np.exp(0.5 * (gammaln(n + 1) - gammaln(m + 1)))
            * (1.0j * u) ** (m - n)
            * genlaguerre(n, m - n)(u**2)
        )
    else:
        matrix[m, n] = (
            np.exp(-u**2 / 2)
            * np.exp(0.5 * (gammaln(m + 1) - gammaln(n + 1)))
            * (1.0j * u) ** (n - m)
            * genlaguerre(m, n - m)(u**2)
        )
return sp.csr_matrix(matrix)

# -----
# Main class
# -----

class PHOTONICChain:
    def __init__(self, N, Omega_C, Chi_C, omega_r, gamma, fock_photon,
                 Omega_J, Chi_J, PBC=False, coeff_tol=1e-9, zero_tol=1e-12):

        self.N = N
        self.Omega_C = Omega_C
        self.Chi_C = Chi_C
        self.gc1 = gc1 = 0.25 * Omega_C * (1 + Chi_C)
        self.gc2 = gc2 = 0.25 * Omega_C * (1 - Chi_C)
        self.Omega_J = Omega_J
        self.Chi_J = Chi_J
        self.gj1 = gj1 = Omega_J * (1 + Chi_J)
        self.gj2 = gj2 = -Omega_J * (1 - Chi_J)
        self.omega_r = omega_r
        self.gamma = gamma
        self.fock_photon = fock_photon
        self.comp_engine = CompositeEngine()
        n = 2 * N

        # -----
        # Capacitive Nambu matrices (identical to ssh_chain_single)
        # -----
        T = np.zeros((n, n))
        for j in range(n - 1):

```

```

        val = gc1 if j % 2 == 0 else gc2
        T[j, j+1] = T[j+1, j] = val
    if PBC and N != 1:
        T[0, n-1] = T[n-1, 0] = gc2

A = 0.5 * (np.eye(n) + T)
B = -0.5 * T

D_herm = np.block([[A, B],
                    [B, A]]) # real symmetric

# -----
# Josephson Nambu matrices (identical to ssh_chain_single)
# -----
TJ = np.zeros((n, n))
for j in range(n - 1):
    val = gj1 if j % 2 == 0 else gj2
    TJ[j, j+1] = TJ[j+1, j] = val
if PBC and N != 1:
    TJ[0, n-1] = TJ[n-1, 0] = gj2

AJ = 0.5 * ((gj1 + gj2) * np.eye(n) - TJ)
BJ = -0.5 * TJ - 0.5 * (gj1 + gj2) * np.eye(n)

H_J_herm = np.block([[AJ, BJ],
                      [BJ, AJ]])

# -----
# Bogoliubov diagonalization via sq_engine
# diagonalize_quadratic expects  $H = A b^\dagger b + B_{\text{half}} b^\dagger b^\dagger$  with
#  $B_{\text{half}} = B/2$  (it internally rescales). Build a minimal operator
# carrying only the (1,0) and (1,1) blocks.
# -----
engine = SecondQuantizationEngine(
    n_modes=n, statistics='boson',
    coeff_tol=coeff_tol, zero_tol=zero_tol
)
self.engine = engine

H_C_for_diag = QuantumOperator([
    TensorTerm(A, (1, 0)),
    TensorTerm(B / 2.0, (1, 1)),
])
_, self.U, self.V = engine.diagonalize_quadratic(H_C_for_diag)

# Reconstruct the full pseudo-unitary P (same structure as ssh_chain_single)

```

```

u_part = self.U # shape (n, n)
v_part = self.V # shape (n, n)
U_V = np.vstack([u_part, v_part])
J_U_V = np.vstack([v_part, u_part]) # J acts as swap for real vecs

self.P = np.hstack([U_V, J_U_V])
self.inverse_P = np.linalg.solve(self.P, np.eye(self.P.shape[0]))

# -----
# Conjugate both Hamiltonians into the Bogoliubov basis
# (exactly as ssh_chain_single does with P.T.conj() @ M @ P)
# -----
self.H_C = self.P.T.conj() @ D_herm @ self.P
self.H_J = self.P.T.conj() @ H_J_herm @ self.P

# -----
# Fock-space builders - direct replicas of ssh_chain_single
# -----
def set_basis(self, states):
    """Registers the explicit Hilbert space bases for the engines."""
    # Must be a numpy array because schrieffer_wolff_correction
    # relies on matrix multiplication: (self.fock_states @ energies_vec)
    self.fock_states = np.array(states)
    self.fock_dim = len(states)
    self.fock_state_index = {tuple(s): i for i, s in enumerate(states)}

def build_HC_fock(self, states, n_total):
    """Diagonal capacitive Hamiltonian in the fixed-excitation Fock basis.

    H_C is diagonal in the Bogoliubov basis; its positive-branch
    eigenvalues sit in the first 2N diagonal entries, with a factor of 2
    from the Bogoliubov transformation convention.
    No zero-point energy / commutation constants included.
    """
    n = 2 * self.N
    energies = np.diag(self.H_C)[:n].real # (2N,) positive-branch energies

    diag_vals = states @ energies # (n_total,) - one energy per state
    mask = np.abs(diag_vals) > self.engine.zero_tol
    idx = np.where(mask)[0]

    return 2.0 * sp.csr_matrix(
        (diag_vals[idx], (idx, idx)), shape=(n_total, n_total)
    )

def build_HJ_fock(self, states, state_index, n_total):

```

```

        """Number-conserving Josephson Hamiltonian in the fixed-excitation Fock basis.

        sum_ij A_J[i,j] b_i^dag b_j + sum_ij D_J[i,j] b_j^dag b_i is a single
        (1,0)-type quadratic operator with combined coefficient
        (A_J + D_J.T) -- relabeling i<->j in the D_J term shows it is exactly
        a (1,0) tensor equal to D_J.T. Built and applied via the engine
        instead of a hand-rolled double loop with manual sqrt bookkeeping.
        The  $\pm 2$ -excitation B/C blocks have no matrix elements within a
        fixed-excitation sector and are correctly absent from this operator.
        """
        n = 2 * self.N
        A_J = self.H_J[:n, :n]
        D_J = self.H_J[n:, n:]

        basis = [tuple(s) for s in states]
        op = QuantumOperator([TensorTerm(A_J + D_J.T, (1, 0))])
        return self.engine.build_sparse_operator_matrix(op, basis)

# -----

def matter_hamiltonians(self, n_excitations, returns_subspace=False):
    if n_excitations < 0:
        raise ValueError("n_excitations cannot be negative")

    basis = self.engine.generate_basis(n_excitations)
    arr = np.array([np.array(s) for s in basis])
    states_indexes = {s: i for i, s in enumerate(basis)}
    n_total = len(basis)

    if returns_subspace:
        H_C_fock = self.build_HC_fock(arr, n_total)
        H_J_fock = self.build_HJ_fock(arr, states_indexes, n_total)
        return arr, states_indexes, n_total, H_C_fock, H_J_fock
    else:
        self.fock_states = arr
        self.fock_state_index = states_indexes
        self.fock_dim = n_total
        self.H_C_fock = self.build_HC_fock(arr, n_total)
        self.H_J_fock = self.build_HJ_fock(arr, states_indexes, n_total)

# -----
# Engine-Powered Tensor Algebra Replaced Below
# -----

def build_full_hamiltonian(self, linear_coupling=True, sw=False):
    H_ph = self.omega_r * _num(self.fock_photon)

```

```

if linear_coupling:
    sin_op = self.gamma * (_destroy(self.fock_photon) + _destroy(self.fock_photon)).
else:
    sin_op = -0.5j * (
        compute_position_exponential_sparse(self.fock_photon, self.gamma)
        - compute_position_exponential_sparse(self.fock_photon, -self.gamma)
    )

self.sin_op = sin_op.toarray() if sp.issparse(sin_op) else np.array(sin_op)

# Delegate tensor products to CompositeEngine
H_ph_full = self.comp_engine.embed_matrix_A(H_ph, self.fock_dim)
H_C_full = self.comp_engine.embed_matrix_B(self.fock_photon, self.H_C_fock)
H_int = self.comp_engine.kron_matrices(sp.csr_matrix(self.sin_op), self.H_J_fock)

self.H = H_ph_full + H_C_full - H_int

if sw:
    self.schrieffer_wolff_correction()
    self.H = self.H + self.H_SW

def decompose_tensor_state(self, full_vector, threshold=0.99):
    basis_A = list(range(self.fock_photon))
    basis_B = [tuple(s) for s in self.fock_states]

    # Engine maps the vector to dictionary of Superpositions
    state_dict = self.comp_engine.vector_to_subsystems(full_vector, basis_A, basis_B, th

    # Unpack back to raw dictionaries and apply the physical thresholding logic
    decomposition = {}
    for alpha, sup in state_dict.items():
        sorted_items = sorted(sup.states.items(), key=lambda x: abs(x[1])**2, reverse=True)
        filtered, cumulative = {}, 0.0
        for key, val in sorted_items:
            filtered[key] = val
            cumulative += abs(val)**2
            if cumulative >= threshold:
                break
        if filtered:
            decomposition[alpha] = filtered

    return decomposition

def compose_tensor_state(self, decomposition):
    basis_A = list(range(self.fock_photon))

```

```

basis_B = [tuple(s) for s in self.fock_states]

# Pack raw dicts into Superpositions for the engine
state_dict = {alpha: Superposition(matter_dict) for alpha, matter_dict in decomposition}

psi = self.comp_engine.subsystems_to_vector(state_dict, basis_A, basis_B)

# Return legacy signature for compatibility with other methods
return psi, list(basis_B), self.fock_state_index

def partial_trace(self, full_vector, which_to_keep='matter'):
    keep_flag = 'B' if which_to_keep == 'matter' else 'A'
    return self.comp_engine.partial_trace(
        full_vector,
        dim_A=self.fock_photon,
        dim_B=self.fock_dim,
        keep=keep_flag
    )

def schrieffer_wolff_correction(self, eps_denom=1e-6):
    """Second-order SW correction - identical logic to ssh_chain_single."""
    n = 2 * self.N
    d_X = self.fock_dim
    d_ph = self.fock_photon
    omega_r = self.omega_r

    n_exc_X = int(self.fock_states[0].sum())
    energies_vec = 2.0 * np.diag(self.H_C[:n]).real # (2N,)
    E_X = self.fock_states @ energies_vec # (d_X,)

    B_J = self.H_J[:n, n:] # b†_i b†_j (+2 sector)
    C_J = self.H_J[n:, :n] # b_i b_j (-2 sector)

    sin_op = self.sin_op

    def build_V_r(states_r, index_r, d_r, H_block, states_X, exc_direction):
        """V_r[r, c] = <r| op |X_c>, where op is the (1,1) creation-pair
        operator (H_block = B_J, exc_direction=+1) or the (0,0)
        annihilation-pair operator (H_block = C_J, exc_direction=-1).
        Built via the engine's rectangular two-basis matrix routine
        instead of two hand-rolled loops with manual sqrt bookkeeping.
        """
        daggers = (1, 1) if exc_direction == +1 else (0, 0)
        op = QuantumOperator([TensorTerm(H_block, daggers)])
        ket_basis = [tuple(s) for s in states_X]
        bra_basis = list(index_r.keys()) # already ordered by index_r construction

```

```

        V = self.engine.build_rectangular_operator_matrix(op, ket_basis, bra_basis)
        return V.toarray()

H_SW = np.zeros((d_ph * d_X, d_ph * d_X), dtype=complex)

for r_sign, exc_delta, H_block in [(+1, +2, B_J), (-1, -2, C_J)]:
    n_exc_r = n_exc_X + exc_delta
    if n_exc_r < 0:
        continue

    states_r_arr = np.array([np.array(c) for c in compositions(n, n_exc_r)])
    index_r      = {tuple(s): i for i, s in enumerate(states_r_arr)}
    d_r          = len(states_r_arr)

    E_r = states_r_arr @ energies_vec

    V_r = build_V_r(states_r_arr, index_r, d_r, H_block, self.fock_states, r_sign)

    for n1 in range(d_ph):
        for n2 in range(d_ph):
            for n_mid in range(d_ph):
                s_factor = sin_op[n1, n_mid] * sin_op[n_mid, n2]
                if abs(s_factor) < 1e-15:
                    continue

                denom1 = (n1 - n_mid) * omega_r + E_X[:, None] - E_r[None, :]
                denom2 = (n_mid - n2) * omega_r + E_r[:, None] - E_X[None, :]

                safe1 = np.abs(denom1) > eps_denom
                safe2 = np.abs(denom2) > eps_denom

                inv1 = np.where(safe1, 1.0 / np.where(safe1, denom1, 1.0), 0.0)
                inv2 = np.where(safe2, 1.0 / np.where(safe2, denom2, 1.0), 0.0)

                term1 = (V_r * inv1.T).T @ V_r.conj()
                term2 = V_r.T @ (V_r.conj() * inv2)

                block = s_factor * (term1 - term2)

                H_SW[n1 * d_X:(n1 + 1) * d_X,
                    n2 * d_X:(n2 + 1) * d_X] += block

H_SW = 0.5 * (H_SW + H_SW.conj().T)
self.H_SW = 0.5 * sp.csr_matrix(H_SW)

# -----

```

```

# Spectrum, vacua, and basis transforms
# -----

def get_low_lying_spectrum(self, k=5):
    """Lowest k eigenvalues/eigenvectors of the full Hamiltonian. Pure
    scipy.sparse.linalg wrapper -- nothing here is a second-quantization
    operation, so there is no engine equivalent to refactor against."""
    H = 0.5 * (self.H + self.H.conj().T)
    try:
        evals, evects = sla.eigsh(H, k=k, which='SA')
    except sla.ArpackNoConvergence as e:
        print(f"Warning: ARPACK did not converge. Returning the {len(e.eigenvalues)} for
        evals, evects = e.eigenvalues, e.eigenvectors
    idx = np.argsort(evals)
    return evals[idx], evects[:, idx]

def build_squeezed_vacuum(self, truncation, threshold=0.99):
    """Analytic squeezed vacuum in the old (bare) Fock basis, expressed
    as a series in the Bogoliubov U, V coefficients. Delegates entirely
    to engine.build_squeezed_vacuum, which reconstructs the same P/P_inv
    and M = solve(U_tilde, V_tilde) this used to build by hand -- verified
    bit-identical to the manual self.inverse_P-based construction."""
    sup = self.engine.build_squeezed_vacuum(self.U, self.V, truncation, threshold)
    self.vacuum = {k: v for k, v in sup.states.items()}
    self.vacuum_n_states = len(self.vacuum)

def build_unsqueezed_vacuum(self, truncation, threshold=0.99):
    """Same series expansion, run on the *inverse* Bogoliubov transform's
    own (U, V) data -- i.e. self.inverse_P's own upper-left/upper-right
    blocks fed back into the same machinery. Verified numerically that
    this reproduces the manual M = solve(self.P[:n,:n], self.P[:n,n:])
    construction exactly."""
    n = 2 * self.N
    U_inv = self.inverse_P[:n, :n]
    V_inv = self.inverse_P[:n, n:]
    sup = self.engine.build_squeezed_vacuum(U_inv, V_inv, truncation, threshold)
    self.unsqueezed_vacuum = {k: v for k, v in sup.states.items()}
    self.unsqueezed_vacuum_n_states = len(self.unsqueezed_vacuum)

def new_to_old_fock(self, new_fock_state, threshold=0.99):
    """Bogoliubov (new) basis Fock state -> bare (old) basis decomposition.

    Equivalent to repeatedly applying  $\eta_i^{\dagger} = \sum_j (P[j,i] b_j^{\dagger} - P[n+j,i] b_j)$  for each occupied mode i, starting from the squeezed
    vacuum self.vacuum. In transform_state's per-mode-row convention this
    is  $U = P[:n,:n].T$ ,  $V = -P[n:,:n].T$  (verified numerically to match the

```

```

original apply_eta_dag exactly, mode by mode, on the vacuum).
"""
n = 2 * self.N
if not hasattr(self, 'vacuum'):
    raise RuntimeError("Call build_squeezed_vacuum() before new_to_old_fock().")

base_state = Superposition({k: complex(v) for k, v in self.vacuum.items()})
U_t = self.P[:n, :n].T
V_t = -self.P[n:, :n].T
state = Superposition({tuple(int(x) for x in new_fock_state): 1.0 + 0j})

result = self.engine.transform_state(state, U_t, V_t, direction='new_to_old',
                                     base_state=base_state)
return self.engine.trim_state(result.states, threshold=threshold)

def old_to_new_fock(self, old_fock_state, threshold=0.99):
    """Bare (old) basis Fock state -> Bogoliubov (new) basis decomposition.

    Equivalent to repeatedly applying  $b_i^{\dagger} = \sum_j (P[i,j] \eta_j^{\dagger} + P[i,n+j] \eta_j)$  for each occupied mode  $i$ , starting from the
    unsqueezed vacuum self.unsqueezed_vacuum. Here  $U = \text{self.U} = P[:n, :n]$ ,
 $V = \text{self.V} = P[n:, :n]$  (verified to match apply_b_dag exactly, since
 $P[:n, n:] == P[n:, :n]$  for this real pseudo-unitary  $P$ ).
    """
    if not hasattr(self, 'unsqueezed_vacuum'):
        raise RuntimeError("Call build_unsqueezed_vacuum() before old_to_new_fock().")

    base_state = Superposition({k: complex(v) for k, v in self.unsqueezed_vacuum.items()})
    state = Superposition({tuple(int(x) for x in old_fock_state): 1.0 + 0j})

    result = self.engine.transform_state(state, self.U, self.V, direction='old_to_new',
                                         base_state=base_state)
    return self.engine.trim_state(result.states, threshold=threshold)

# -----
# Quanta distribution / IPR observables (chain-specific physics; no
# generic engine equivalent -- these concern physical site occupation,
# not abstract Fock-space structure)
# -----

def quanta_distribution(self, fock_vector, state_coeff_dict=None):
    n = 2 * self.N
    if isinstance(fock_vector, dict):
        sup = Superposition({k: complex(v) for k, v in fock_vector.items()})
        distribution = self.engine.get_mode_occupations(sup)
    else:

```

```

        weights = np.abs(fock_vector) ** 2
        distribution = weights @ self.fock_states

    total = distribution.sum()
    if total > 1e-15:
        distribution /= total
    return distribution

def quanta_ipr(self, fock_vector):
    weights = np.abs(fock_vector) ** 2
    distribution = weights @ self.fock_states
    total = distribution.sum()
    if total < 1e-15:
        return 0.0, np.inf, distribution
    ipr = np.sum(distribution ** 2) / (total ** 2)
    return ipr, 1.0 / ipr, distribution

def quanta_ipr_mixed(self, rho_matter):
    diag_rho = np.real(np.diag(rho_matter))
    distribution = diag_rho @ self.fock_states
    total = distribution.sum()
    if total < 1e-15:
        return 0.0, np.inf, distribution
    ipr = np.sum(distribution ** 2) / total ** 2
    return ipr, 1.0 / ipr, distribution

def compute_ipr_batch(self, vecs):
    if not hasattr(self, '_C_matrix'):
        raise RuntimeError("Call warm_old_basis_cache() before compute_ipr_batch().")

    d_ph = self.fock_photon
    d_mat = self.fock_dim
    k = vecs.shape[1]

    C = self._C_matrix
    sorted_states_arr = self._old_states_arr

    vecs_3d = vecs.reshape(d_ph, d_mat, k)
    amp_3d = np.tensordot(C, vecs_3d, axes=([1], [1]))
    amp_3d = amp_3d.transpose(1, 0, 2)

    diag_rho = np.sum(np.abs(amp_3d) ** 2, axis=0)

    all_dist = sorted_states_arr.T @ diag_rho
    totals = all_dist.sum(axis=0)
    iprs = np.where(totals < 1e-15, 0.0, np.sum(all_dist ** 2, axis=0) / totals ** 2)

```

```

        return iprs

def warm_old_basis_cache(self):
    if not hasattr(self, '_new_to_old_cache'):
        self._new_to_old_cache = {}

    for state in self.fock_states:
        key = tuple(state)
        if key not in self._new_to_old_cache:
            self._new_to_old_cache[key] = self.new_to_old_fock(np.array(state, dtype=int))

    all_old_states = set()
    for old_dict in self._new_to_old_cache.values():
        all_old_states.update(old_dict.keys())

    sorted_states = sorted(all_old_states, key=lambda s: (sum(s), s))
    state_to_idx = {s: i for i, s in enumerate(sorted_states)}
    d_mat_old = len(sorted_states)
    d_mat = self.fock_dim

    C = np.zeros((d_mat_old, d_mat), dtype=complex)
    for s_idx, state in enumerate(self.fock_states):
        new_key = tuple(state)
        old_dict = self._new_to_old_cache[new_key]
        norm = np.sqrt(sum(abs(v) ** 2 for v in old_dict.values()))
        if norm < 1e-15:
            continue
        for old_state, coeff in old_dict.items():
            C[state_to_idx[old_state], s_idx] += coeff / norm

    self._C_matrix = C
    self._old_states_arr = np.array(sorted_states, dtype=float)
    self._old_state_idx = state_to_idx

def decompose_to_old_fock(self, full_vector, threshold=0.99):
    bogoliubov_decomp = self.decompose_tensor_state(full_vector, threshold=threshold)

    if not hasattr(self, '_new_to_old_cache'):
        self._new_to_old_cache = {}

    old_decomposition = {}
    for alpha, matter_dict in bogoliubov_decomp.items():
        old_matter = {}
        for new_state_tuple, coeff in matter_dict.items():
            if new_state_tuple not in self._new_to_old_cache:
                self._new_to_old_cache[new_state_tuple] = self.new_to_old_fock(

```

```

        np.array(new_state_tuple, dtype=int), threshold=threshold
    )
    old_part = self._new_to_old_cache[new_state_tuple]
    norm_old = np.sqrt(sum(abs(v) ** 2 for v in old_part.values()))
    for old_state, old_coeff in old_part.items():
        weighted = coeff * old_coeff / (norm_old if norm_old > 1e-15 else 1.0)
        old_matter[old_state] = old_matter.get(old_state, 0.0) + weighted
    if old_matter:
        sorted_items = sorted(old_matter.items(), key=lambda x: abs(x[1]) ** 2, reverse=True)
        filtered, cumulative = {}, 0.0
        total_w = sum(abs(v) ** 2 for v in old_matter.values())
        for key, val in sorted_items:
            filtered[key] = val
            cumulative += abs(val) ** 2
            if cumulative >= threshold * total_w:
                break
        old_decomposition[alpha] = filtered

    return old_decomposition

def build_squeezed_vacuum_tensor(self, alpha=0, truncation=10, threshold=0.99):
    """Builds the Bogoliubov squeezed vacuum directly as a 1D composite vector."""
    vacuum_superpos = self.engine.build_squeezed_vacuum(self.U, self.V, truncation, threshold)
    initial_state = {alpha: vacuum_superpos}

    basis_A = list(range(self.fock_photon))
    basis_B = [tuple(s) for s in self.fock_states]
    return self.comp_engine.subsystems_to_vector(initial_state, basis_A, basis_B)

def build_unsqueezed_vacuum_tensor(self, alpha=0, truncation=10, threshold=0.99):
    """Builds the unsqueezed vacuum directly as a 1D composite vector."""
    n = 2 * self.N
    U_inv = self.inverse_P[:n, :n]
    V_inv = self.inverse_P[:n, n:]

    unsqueezed_superpos = self.engine.build_squeezed_vacuum(U_inv, V_inv, truncation, threshold)
    initial_state = {alpha: unsqueezed_superpos}

    basis_A = list(range(self.fock_photon))
    basis_B = [tuple(s) for s in self.fock_states]
    return self.comp_engine.subsystems_to_vector(initial_state, basis_A, basis_B)

def transform_tensor_to_old(self, full_vector, truncation=2, threshold=0.99):
    """Transforms a full composite vector from quasiparticle to bare operators."""
    basis_A = list(range(self.fock_photon))
    basis_B = [tuple(s) for s in self.fock_states]

```

```

state_dict = self.comp_engine.vector_to_subsystems(full_vector, basis_A, basis_B, th

n = 2 * self.N
U_t = self.P[:n, :n].T
V_t = -self.P[n:, :n].T
# Use the provided truncation parameter
base_vacuum = self.engine.build_squeezed_vacuum(self.U, self.V, truncation=truncati

def map_to_old(superpos):
    return self.engine.transform_state(
        superpos, U_t, V_t, direction='new_to_old', base_state=base_vacuum
    ).trim(threshold)

mapped_dict = self.comp_engine.map_subsystem_B(state_dict, map_to_old)
return self.comp_engine.subsystems_to_vector(mapped_dict, basis_A, basis_B)

def transform_tensor_to_new(self, full_vector, truncation=8, threshold=0.99):
    """Transforms a full composite vector from bare to quasiparticle operators."""
    basis_A = list(range(self.fock_photon))
    basis_B = [tuple(s) for s in self.fock_states]
    state_dict = self.comp_engine.vector_to_subsystems(full_vector, basis_A, basis_B, th

n = 2 * self.N
U_inv = self.inverse_P[:n, :n]
V_inv = self.inverse_P[:n, n:]
# Use the provided truncation parameter
base_vacuum = self.engine.build_squeezed_vacuum(U_inv, V_inv, truncation=truncation)

def map_to_new(superpos):
    return self.engine.transform_state(
        superpos, self.U, self.V, direction='old_to_new', base_state=base_vacuum
    ).trim(threshold)

mapped_dict = self.comp_engine.map_subsystem_B(state_dict, map_to_new)
return self.comp_engine.subsystems_to_vector(mapped_dict, basis_A, basis_B)

```